Advanced Developer Fundamentals

Demonstrate knowledge of localization and multi-currency features and capabilities and how they affect coding.

Table of Contents

After studying this topic, you should be able to:

- Explain the role and importance of localization tools in orgs that work with multiple languages, locales, or currencies.
- Identify best practices when working with a multi-language, multi-locale, or multi-currency Salesforce org.
- Identify common use cases for localization tools in working with Apex or low-code customizations.
- Describe how to retrieve local language, locale, and currency programmatically.

Introduction

Salesforce provides a **number** of **tools** for creating apps that can accommodate **multiple languages**, **currencies**, and **locales**. Object and field label **translations** can be provided declaratively through the user interface by specified users. To ensure that information is displayed correctly to all users regardless of language, locale or preferred currency, developers should use **functions** and **special label notations** provided in **SOQL**, **Visualforce**, and **Apex**.

Tools for Localization

Multi-Currency Tools

CurrencyType object: stores currencies used by org

DatedConversionRate object: stores dated exchange rates

CurrencyIsoCode field: defines which currency is used in a record

convertCurrency() function: performs numeric conversion of a record's currency value into the user's currency value in SOQL/SOSL

FORMAT() function: displays a currency value both in the original record source and in the currency of the current user in SOQL/SOSL

Multi-Language Tools

Translation Workbench & Rename Tabs & Labels screen: Enter translations for objects and fields

{!$ObjectType.[object].fields.name.label}: Displays a translated label in Visualforce

System.Label: accesses translated labels in Apex

toLabel(): translates a label in SOQL/SOSL

Multi-Locale Tools

FORMAT() function: formats number, date, time and currency fields based on user's locale in SOQL/SOSL

<apex:outputField> tag: formats data according to the user's locale

General Guidelines

Table of Contents

General Guidelines

Deploying Salesforce applications to **international communities** requires adapting to variations in **language**, **locale**, and **currency**.

LANGUAGE

Controls **labels** and **translations**

LOCALE

Controls **formatting** of information such as **dates**, **times** and **names**, as well the use of **period** vs. **comma** for thousands and decimal separators

CURRENCY

Controls **currency display** such as the currency symbol and conversion based on defined exchange rates

General Guidelines

DYNAMIC SOQL/APEX

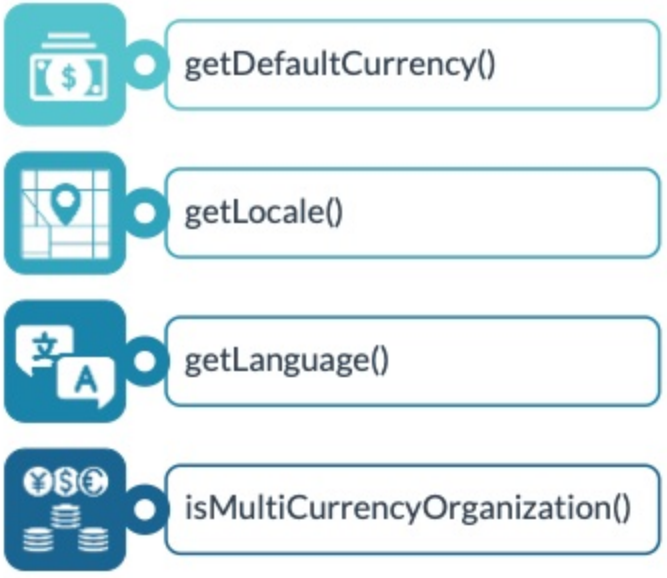
Dynamic SOQL and Apex can be used for code that is **language-dependent**, **locale-dependent**, or **currency-dependent**.

STRINGS AND LENGTHS

Building logic based on literal **string values** or **specific string lengths** in Apex code, formula, etc. **should be avoided** as the logic may break, for example, as a result of a translation when the current language is changed.

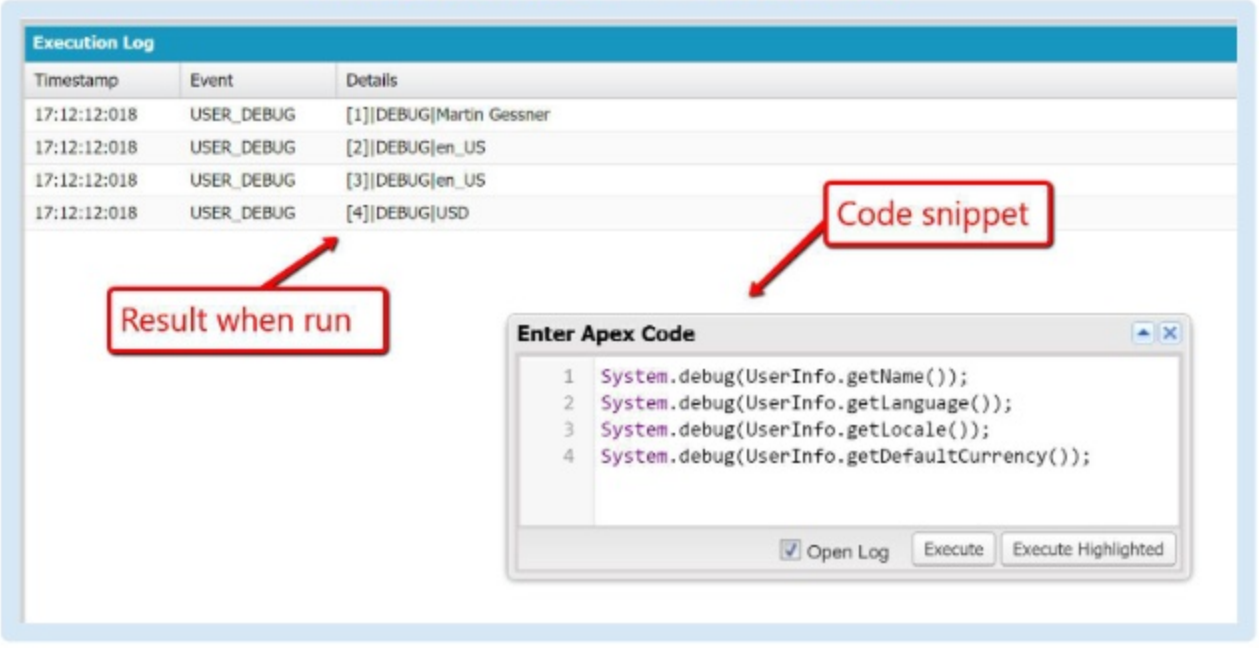
General Guidelines

The **System.UserInfo Class** provides the methods below that can be used for localization:



System.UserInfo Class Example

The Apex code below demonstrates using **UserInfo methods** to determine the **language**, **locale**, and **default currency** of the **current user**.



Multi-language

Table of Contents

Multi-Language Techniques

Salesforce is equipped with **features** and **tools** to provide a **multilingual environment** for its users.

LANGUAGE SUPPORT LEVELS

Salesforce has **three levels** of **language support**: **fully supported, end-user, and platform only**

FULLY SUPPORTED LANGUAGES

All Salesforce features and user interface texts are **automatically translated** into the organization's **default language**.

END-USER LANGUAGES

Labels for all standard objects and pages, **except** admin pages, Setup, and Help, can be **translated** using end-user languages. **Right-to-left** languages are also supported but have some **limitations**.

PLATFORM-ONLY LANGUAGES

Custom **apps** and **functionality** developed in the Salesforce platform can be **localized** using platform-only languages, such as translating **custom** labels, objects, and fields and **most standard** labels, objects, and fields.

Multi-Language Techniques

TEXT FIELD LENGTHS

Text field lengths should be **sufficient** to **accommodate longer translations**. A three-word phrase in English, for example, may be translated as a fifteen-word phrase in Russian.

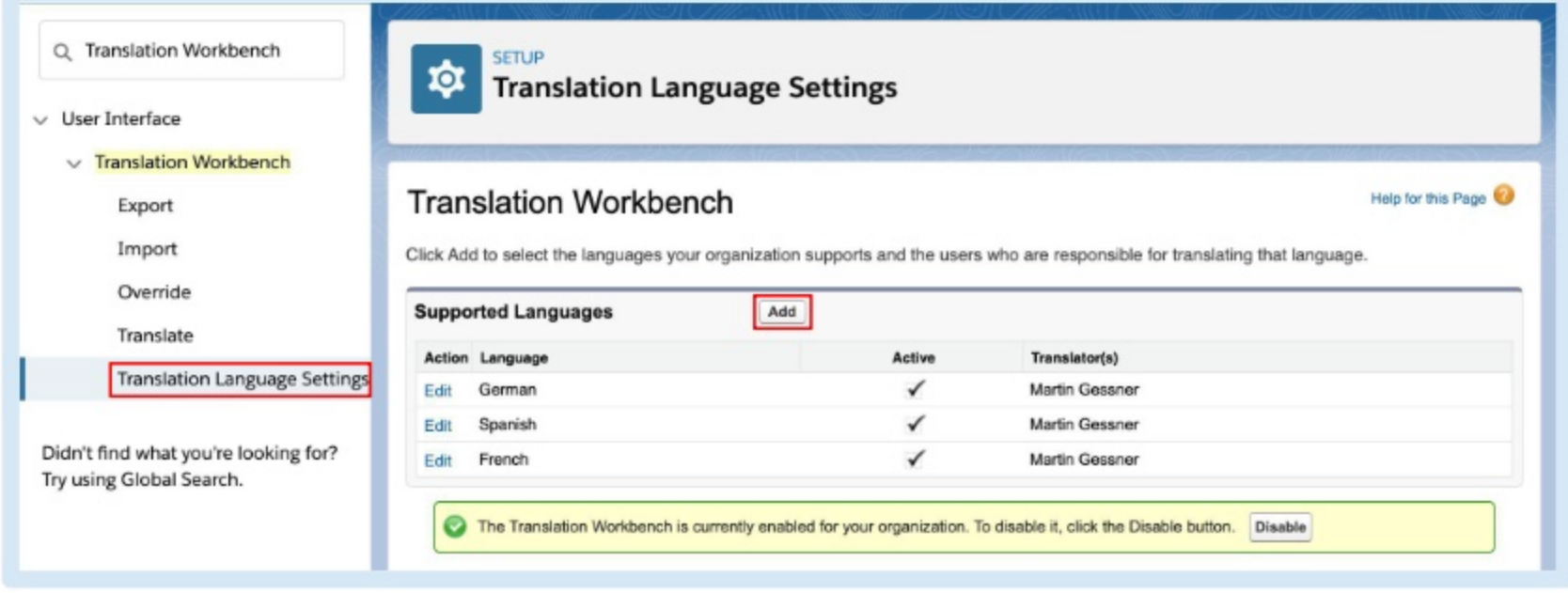
TRANSLATION TOOLS

Use the **Translation Workbench** and the **Rename Tabs and Labels** screens to:

- Translate customizations.
- Change labels in installed **custom managed packages.**

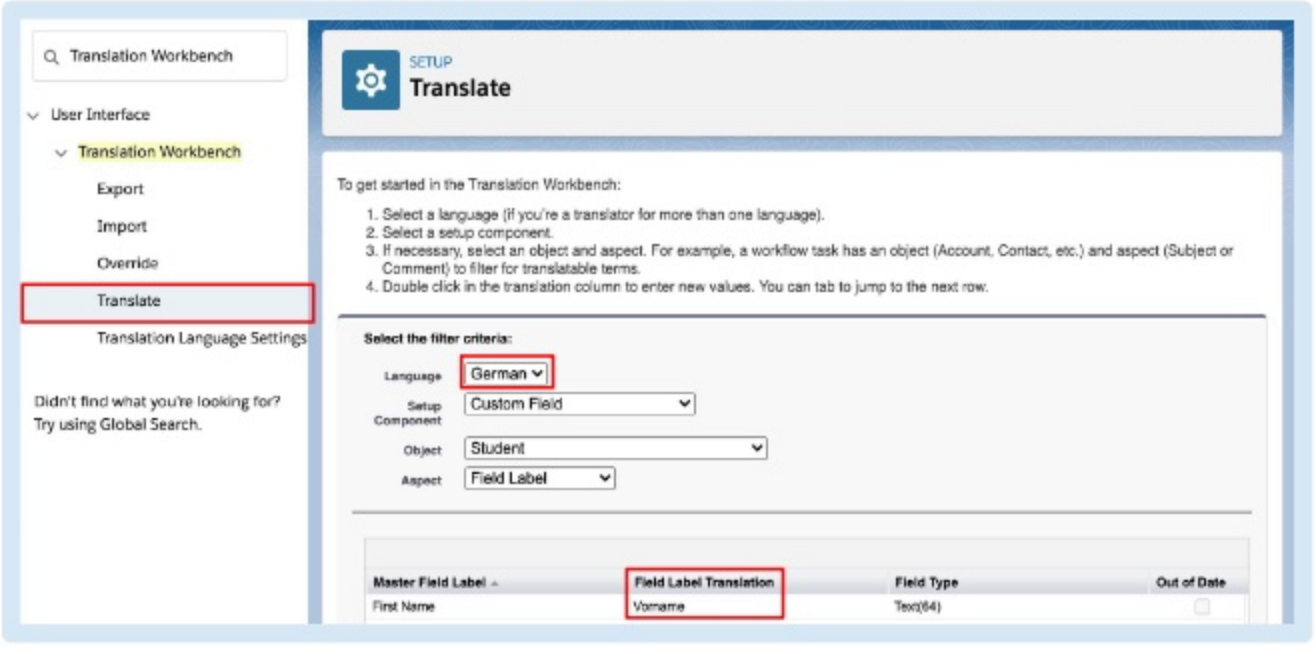
Add Languages in Translation Workbench

The *Translation Workbench* can be used to add **supported languages** and **manage translations** for metadata and data labels in the organization.



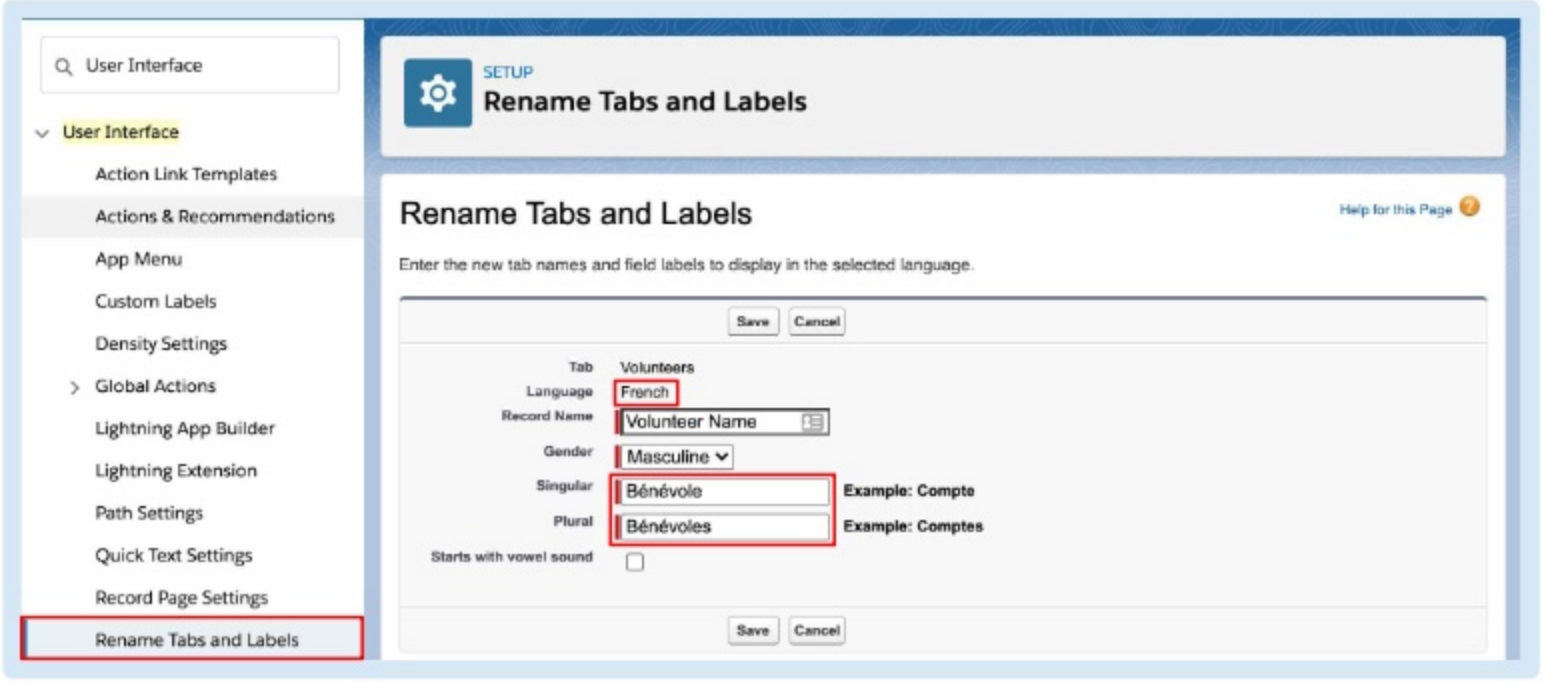
Translate via Translation Workbench

The following example illustrates creating a **translation** for a **field name** of a custom object using the **Translate page** in *Translation Workbench*.



Translate Tabs and Labels

The *Rename Tabs and Labels* page in Setup can be used to **translate tabs** and **labels**.



Multi-Language Techniques

The following are some **recommendations** for building **localization-ready** applications or functionality:

VISUALFORCE PAGE

{!$ObjectType.[object].fields.name.label} can be used to display labels.

SOQL / SOSL

The **toLabel()** function can be used in SOQL or SOSL to translate labels.

APEX

System.Label in Apex can be used to make translation-friendly error messages.

PICKLISTS

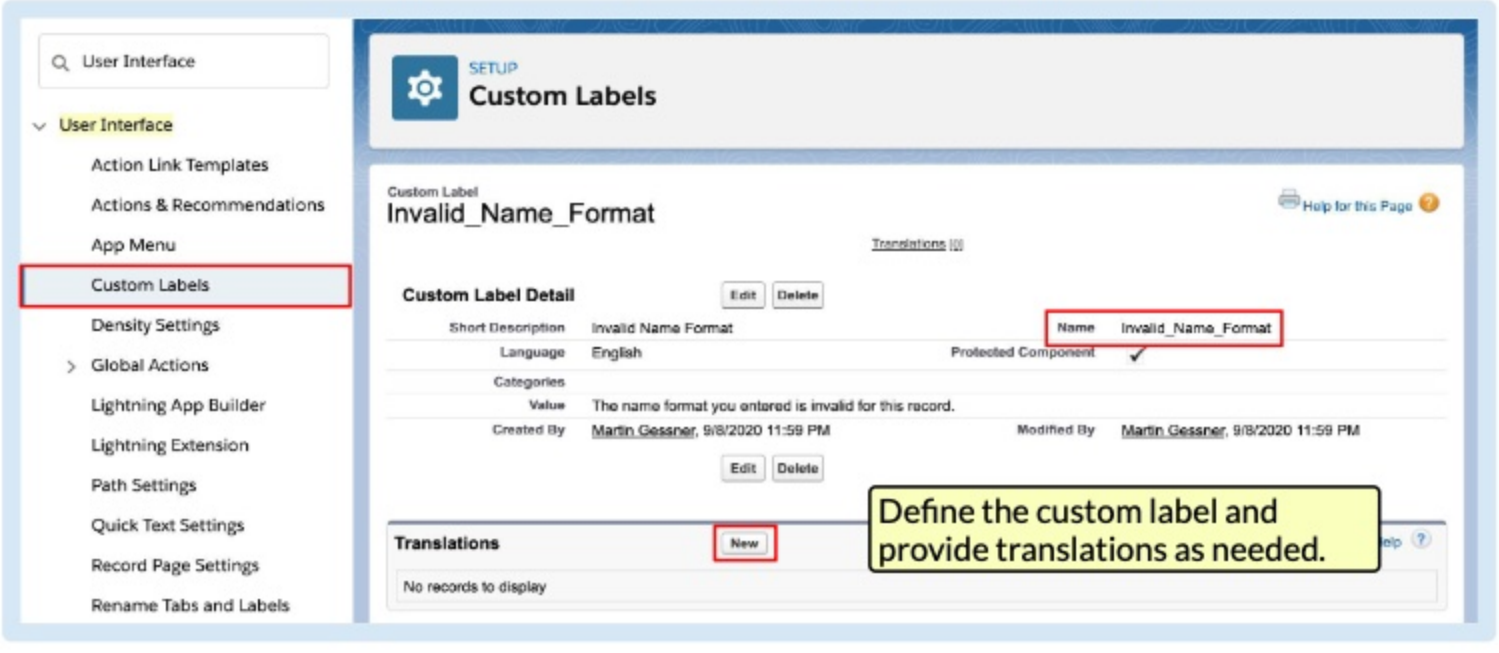
Picklists should be used **whenever possible** instead of allowing users to type in free-form text.

EMAIL TEMPLATES

Use **Visualforce email templates** instead of standard email templates for locale-aware and translation-friendly templates.

Translate Custom Labels

Translations can be created for **custom labels** in the *Custom Labels* page in Setup. In this example, a custom label is used for displaying an error message.



Custom Label in the Code

The Apex code below shows how a **custom label** is used to display a **localized error message** in a Visualforce page.

```
if (isValidNameFormat(name)) {

   processName(name);

} else {

   // show multi-language friendly error message
   ApexPages.addMessage(
       new ApexPages.Message(ApexPages.Severity.ERROR, System.Label.Invalid_Name_Format);
   );
}
```

A custom label is accessed in Apex using *System.Label* to display a localized error message based on the current user's settings.

Accessing Label Dynamically

A **translation** can also be retrieved dynamically by using the **System.Label.get** method.

```
// If namespace is an empty string, it refers to the org namespace
// If namespace is set to null, it refers to the package namespace
String namespace = ''; // if empty

String label = 'OutOfStock';
String language = 'de'; // language ISO code for German

String message = '';

if (System.Label.translationExists(namespace, label, language)) {

   message = System.Label.get(namespace, label, language);

} else {

   // get default translation if not available
   message = System.Label.get(namespace, label);
}
```

The **Label.translationExists** method can be used to check first if the translation exists for the specified language.

Multi-Language Techniques

When building localization-ready applications, **script direction** and **auto-formatting** capabilities are considered.

DIRECTIONALITY

Languages used in the org that are **read left-to-right** and **right-to-left** should be considered when designing **user interfaces**. It can **affect** the layout order of fields and CSS styling.

DATA FORMATTING

Locale-aware components such as **<apex:outputField>** should be used to **automatically format** data based on the underlying field type, such as **currency**, **date**, **checkbox**, **picklist**, etc.

COMPONENT LABELS

In the Lightning App Builder, the expression '**{!$Label.customLabelName}**' can be used to specify localized custom component labels.

Multi-currency

Table of Contents

Multi-Currency

Salesforce offers features to support organizations that use **multiple currencies**. Note that some features are only available when **multi-currency** is **enabled** for the org.

CURRENCY ISO CODE

All objects that have **currency fields** will also have a **CurrencyIsoCode** field, which specifies the currency that the numeric amount represents.

CONVERTCURRENCY

The **convertCurrency()** function can be used in SOQL or SOSL to convert a **currency value** into the current user's currency.

FORMAT

The **FORMAT()** function can be used in SOQL or SOSL to apply a localized formatting of a **currency**, **number**, **date**, or **time**. It can also be used to display the currency value both in the **original record source** and in the **currency of the current user**.

CurrencyIsoCode Field

The following SOQL query **includes** the *CurrencyIsoCode* field of **each record** in the results.

```
SELECT Name, CurrencyIsoCode, Purchase_Cost__c FROM Bike_Part__c
```

Query Results - Total Rows: 3

Name	CurrencyIsoCode	Purchase_Cost__c
Sprocket - 10 speed	USD	130.5
Fork - Australian Assembly	AUD	275.66
Handlebars - European Assembly	EUR	163.75

Query Grid: Save Rows | Insert Row | Delete Row | Refresh Grid

Access in Salesforce: Create New | Open Detail Page | Edit Page

CurrencyType Object

The **CurrencyType** object exists in orgs where the multiple currencies feature is enabled and is used to **store the currencies** being added or used in the org.

SELECT ConversionRate, DecimalPlaces, IsActive, IsCorporate, IsoCode FROM CurrencyType

Query Results - Total Rows: 3

ConversionRate	DecimalPlaces	IsActive	IsCorporate	IsoCode
1.38	2	true	false	AUD
0.88	2	true	false	EUR
1	2	true	true	USD

Query Grid: Save Rows | Insert Row | Delete Row | Refresh Grid

Access in Salesforce: Create New | Open Detail Page | Edit Page

DatedConversionRate Object

The **DatedConversionRate** object stores dated exchange rates. **Dated exchange rates** is a feature that allows **applying conversion rates** in the org based on a specific date range.

```
SELECT StartDate, NextStartDate, IsoCode, ConversionRate FROM DatedConversionRate ORDER BY StartDate ASC
```

Query Results - Total Rows: 5

StartDate	NextStartDate	IsoCode	ConversionRate
0000-12-30	2021-01-01	AUD	1.38
0000-12-30	2021-01-01	EUR	0.88
0000-12-30	9999-12-31	USD	1
2021-01-01	9999-12-31	AUD	1.35
2021-01-01	9999-12-31	EUR	0.89

Query Grid: Save Rows | Insert Row | Delete Row | Refresh Grid

Access in Salesforce: Create New | Open Detail Page | Edit Page

Format Function

When using the **format function** on a **currency** field, the value is formatted according to the associated **currency ISO code** per record. A **converted value** based on the user's currency is also displayed.

```
SELECT Name, format(Purchase_Cost__c) FROM Bike_Part__c
```

Query Results - Total Rows: 3

Name	Purchase_Cost__c
Sprocket - 10 speed	USD 130.50
Fork - Australian Assembly	AUD 275.66 (USD 199.75)
Handlebars - European Assembly	EUR 163.75 (USD 186.08)

Query Grid: Save Rows | Insert Row | Delete Row | Refresh Grid

Access in Salesforce: Create New | Open Detail Page | Edit Page

Convert and Format Function

The **convertCurrency** function is used to **convert** the numeric amount into the **user's currency**. The **format** function is used to properly format the value as a **currency amount**.

```
SELECT Name, format(convertCurrency(Purchase_Cost__c)) FROM Bike_Part__c
```

Query Results - Total Rows: 3

Name	Purchase_Cost__c
Sprocket - 10 speed	USD 130.50
Fork - Australian Assembly	USD 199.75
Handlebars - European Assembly	USD 186.08

Query Grid: Save Rows | Insert Row | Delete Row | Refresh Grid

Access in Salesforce: Create New | Open Detail Page | Edit Page

Handle Multiple Currencies

Visualforce pages can be built to support **multi-currency** requirements, such as using the **outputField** component to **automatically format** currency data.

```
<!-- the opportunity amount will automatically format based on the user's settings-->
<apex:outputField value="{! Opportunity.Amount }"/>
```

Note, however, that the ***outputField*** component **cannot be used** to display currency values when *Advanced Currency Management* is enabled in the org. In this case, the ***outputText*** component can be used.

Localization in Lightning Web Components

Lightning web components can be configured adapt to user **languages**, **currencies**, and **timezones** by using the **@salesforce/i18n** scoped module.

```
// LocalExample.js.
import { LightningElement } from "lwc";
import LOCALE from "@salesforce/i18n/locale";
import CURRENCY from "@salesforce/i18n/currency";
export default class LocalExample extends LightningElement {
    myLocale = LOCALE;
    myCurrency = CURRENCY;
    localDate;
    localPrice;

    connectedCallback() {

        let date = new Date(2024, 1, 14);
        this.localDate = new Intl.DateTimeFormat(LOCALE).format(date);
        let price = 11.99;
        let setting = Intl.NumberFormat(LOCALE,
          { style: 'currency', currency: CURRENCY });
        this.localPrice = setting.format(price);
    }
}
```

```
<!-- LocalExample.html -->
<template>
    <div class="slds-color__background_gray-1 slds-box">
        My locale is {myLocale}.<br />
        The local date is {localDate}.<br />
        My currency is {myCurrency}.<br />
        The price is {localPrice}
    </div>
</template>
```

This example shows how the component can display information based on the the user's current **locale** and **currency settings**.

My locale is en-PH.
The local date is 2/14/2024.
My currency is AUD.
The price is A$11.99

Learn More

- Support Users in Multiple Languages
- Localization
- Supported Languages
- Querying Currency Fields in Multi-Currency Orgs
- Translate Returned SOQL Results
- Access Internationalization Properties